PSTAT 131 HW 3

Evan Jamison

2025-11-21

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.0     v tibble    3.2.1
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(ISLR)
library(glmnet)
```

```
## Loading required package: Matrix
##
## Attaching package: 'Matrix'
##
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
##
## Loaded glmnet 4.1-8
```

```
library(tree)
library(maptree)
```

```
## Loading required package: cluster
## Loading required package: rpart
```

```
library(randomForest)
```

```
## randomForest 4.7-1.2
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
##
```

```
## The following object is masked from 'package:dplyr':
##
##     combine
##
## The following object is masked from 'package:ggplot2':
##
##     margin
```

```
library(gbm)
```

```
## Loaded gbm 2.2.2
## This version of gbm is no longer under development. Consider transitioning to gbm3, https://github.co
```

```
library(ROCR)
```

Predicting carseats sales using regularized regression methods

```
set.seed(123)
dat <- model.matrix(Sales~., Carseats)
train = sample(nrow(dat), 50)
x.train = dat[train, ]
y.train = Carseats[train, ]$Sales
# The rest as test data
x.test = dat[-train, ]
y.test = Carseats[-train, ]$Sales
```

a.

```
lambda.list.ridge = 1000 * exp(seq(0, log(1e-5), length = 100))
ridge = cv.glmnet(
  x.train,
  y.train,
  alpha = 0,
  lambda = lambda.list.ridge,
  nfolds = 5
)

blr = ridge$lambda.min
blr
```

```
## [1] 0.02009233
```

```
ridge_final = glmnet(
  x.train,
  y.train,
  alpha = 0,
  lambda = blr
)

coef(ridge_final)
```

```
## 13 x 1 sparse Matrix of class "dgCMatrix"
##                              s0
## (Intercept)      4.444433369
## (Intercept)      .
## CompPrice        0.115271251
## Income           0.020223213
## Advertising      0.113779840
## Population       0.001314488
## Price           -0.104559122
## ShelveLocGood    4.899415173
## ShelveLocMedium  2.021132705
## Age             -0.059240816
## Education       -0.067153028
## UrbanYes         0.201518225
## USYes            0.125211905
```

b.

```
ridge_train_pred = predict(ridge_final, newx = x.train)
training_mse = mean((ridge_train_pred - y.train)^2)
training_mse
```

```
## [1] 0.5715764
```

```
ridge_test_pred = predict(ridge_final, newx = x.test)
test_mse = mean((ridge_test_pred - y.test)^2)
test_mse
```

```
## [1] 1.293105
```

I find that the test MSE is significantly higher than the training MSE. However, this is common/normal behavior.

c.

```
lambda.list.lasso = 2 * exp(seq(0, log(1e-4), length = 100))

lasso = cv.glmnet(
  x.train,
  y.train,
  alpha = 1,
  lambda = lambda.list.lasso,
  nfolds = 10
)

bll = lasso$lambda.min
bll
```

```
## [1] 2e-04
```

```
lasso_final = glmnet(
  x.train,
  y.train,
  alpha = 1,
  lambda = bll
)

coef(lasso_final)
```

```
## 13 x 1 sparse Matrix of class "dgCMatrix"
##                              s0
## (Intercept)      4.247308831
## (Intercept)      .
## CompPrice        0.120085847
## Income           0.020274809
## Advertising      0.113412113
## Population       0.001443082
## Price           -0.108152314
## ShelveLocGood    4.923291722
## ShelveLocMedium  2.050110836
## Age             -0.061406786
## Education       -0.064686301
## UrbanYes         0.219132392
## USYes            0.176107079
```

the lasso model did shrink many coefficients towards 0 (some very close) but only USYes to 0. This means the lasso penalty was not strong enough to completely neutralize the predictors.

d.

```
lasso_train_pred = predict(lasso_final, newx = x.train)
lasso_train_mse = mean((lasso_train_pred - y.train)^2)
lasso_train_mse
```

```
## [1] 0.5680636
```

```
lasso_test_pred = predict(lasso_final, newx = x.test)
lasso_test_mse = mean((lasso_test_pred - y.test)^2)
lasso_test_mse
```

```
## [1] 1.368444
```

I find that similarly to ridge regression, the test MSE is significantly higher than the training MSE.

e.

Both Ridge and Lasso in this instance behaved quite similarly, both producing very similar training and test mses, with ridge performing just slightly better. Both lambda values chosen by each model were very small, however the lasso model produced a smaller lambda. Generally, ridge only shrinks coeficients while lasso while often completely set some to zero, but perhaps since the trianing size was only 50, the lasso model did not get rid of any predictors so the two models performed very similarly.

Using bootstrap to compute uncertainty about a parameter of interes

```r
shots  <- 784
makes  <- 311
B <- 10000
shots_total <- c(rep(1, makes), rep(0, shots - makes))

bootstrap_fg <- replicate(B, {
  mean(sample(shots, size = shots, replace = TRUE))
})

ci_99 <- quantile(bootstrap_fg, prob = c(0.005, 0.995))
ci_99
```

```
##     0.5%    99.5%
## 371.7915 412.1736
```

Analyzing drug use

a.

```r
setwd("C:/Users/user/OneDrive/Documents")

drug <- read_csv('drug.csv',
                 col_names = c('ID','Age','Gender','Education','Country',
                 'Ethnicity','Nscore','Escore','Oscore','Ascore','Cscore',
                 'Impulsive','SS','Alcohol','Amphet','Amyl','Benzos',
                 'Caff','Cannabis','Choc','Coke','Crack','Ecstasy',
                 'Heroin','Ketamine','LegalH','LSD','Meth','Mushrooms',
                 'Nicotine','Semer','VSA')
)
```

```
## Rows: 1885 Columns: 32
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (19): Alcohol, Amphet, Amyl, Benzos, Caff, Cannabis, Choc, Coke, Crack, ...
## dbl (13): ID, Age, Gender, Education, Country, Ethnicity, Nscore, Escore, Os...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
drug <- drug %>%
  mutate(
    recent_nicotine_use = ifelse(Nicotine >= "CL3", "Yes", "No"),
    recent_nicotine_use = factor(recent_nicotine_use)
  )
head(drug)
```

```
## # A tibble: 6 x 33
##      ID     Age Gender Education Country Ethnicity Nscore Escore  Oscore Ascore
##   <dbl>   <dbl>  <dbl>     <dbl>   <dbl>     <dbl>  <dbl>  <dbl>   <dbl>  <dbl>
## 1     1  0.498   0.482   -0.0592   0.961     0.126  0.313 -0.575 -0.583  -0.917
## 2     2 -0.0785 -0.482    1.98     0.961    -0.317 -0.678  1.94   1.44    0.761
```

```
## 3     3  0.498  -0.482   -0.0592   0.961    -0.317 -0.467  0.805 -0.847  -1.62
## 4     4 -0.952   0.482    1.16     0.961    -0.317 -0.149 -0.806 -0.0193  0.590
## 5     5  0.498   0.482    1.98     0.961    -0.317  0.735 -1.63  -0.452  -0.302
## 6     6  2.59    0.482   -1.23     0.249    -0.317 -0.678 -0.300 -1.56    2.04
## # i 23 more variables: Cscore <dbl>, Impulsive <dbl>, SS <dbl>, Alcohol <chr>,
## #   Amphet <chr>, Amyl <chr>, Benzos <chr>, Caff <chr>, Cannabis <chr>,
## #   Choc <chr>, Coke <chr>, Crack <chr>, Ecstasy <chr>, Heroin <chr>,
## #   Ketamine <chr>, LegalH <chr>, LSD <chr>, Meth <chr>, Mushrooms <chr>,
## #   Nicotine <chr>, Semer <chr>, VSA <chr>, recent_nicotine_use <fct>
```

b.

```
drug_sub <- drug %>%
  select(Age:SS, recent_nicotine_use)
head(drug_sub)
```

```
## # A tibble: 6 x 13
##       Age Gender Education Country Ethnicity Nscore Escore  Oscore Ascore
##     <dbl>  <dbl>     <dbl>   <dbl>     <dbl>  <dbl>  <dbl>   <dbl>  <dbl>
## 1  0.498   0.482   -0.0592   0.961     0.126  0.313 -0.575 -0.583  -0.917
## 2 -0.0785 -0.482    1.98     0.961    -0.317 -0.678  1.94   1.44    0.761
## 3  0.498  -0.482   -0.0592   0.961    -0.317 -0.467  0.805 -0.847  -1.62
## 4 -0.952   0.482    1.16     0.961    -0.317 -0.149 -0.806 -0.0193  0.590
## 5  0.498   0.482    1.98     0.961    -0.317  0.735 -1.63  -0.452  -0.302
## 6  2.59    0.482   -1.23     0.249    -0.317 -0.678 -0.300 -1.56    2.04
## # i 4 more variables: Cscore <dbl>, Impulsive <dbl>, SS <dbl>,
## #   recent_nicotine_use <fct>
```

c.

```
train_idx <- sample(nrow(drug_sub), 1000)
train <- drug_sub[train_idx, ]
test  <- drug_sub[-train_idx, ]
```

d.

```
logit_reg <- glm(recent_nicotine_use ~ .,
                 data = train,
                 family = "binomial")
summary(logit_reg)
```

```
##
## Call:
## glm(formula = recent_nicotine_use ~ ., family = "binomial", data = train)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.62389    0.16007   3.898 9.72e-05 ***
## Age         -0.41329    0.08901  -4.643 3.43e-06 ***
## Gender      -0.36922    0.16431  -2.247  0.02464 *
## Education   -0.36379    0.08011  -4.541 5.59e-06 ***
```

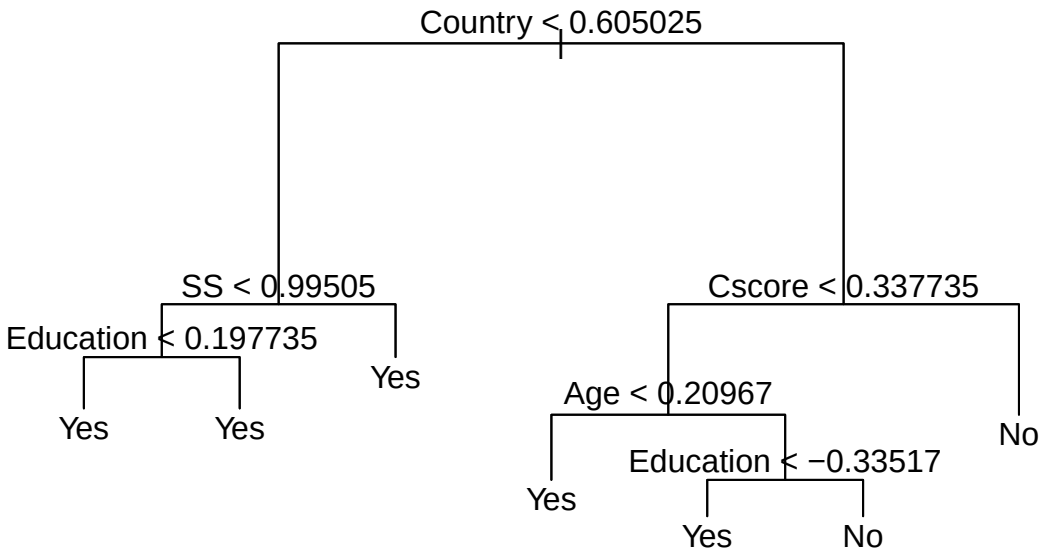
```
## Country     -0.39061    0.12014  -3.251  0.00115 **
## Ethnicity    0.34614    0.45343   0.763  0.44524
## Nscore       0.11958    0.08964   1.334  0.18220
## Escore      -0.05560    0.09351  -0.595  0.55208
## Oscore       0.17454    0.08917   1.957  0.05030 .
## Ascore       0.04561    0.07821   0.583  0.55979
## Cscore      -0.24886    0.08946  -2.782  0.00540 **
## Impulsive   -0.02177    0.10072  -0.216  0.82888
## SS           0.31810    0.11012   2.889  0.00387 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 1364.9  on 999  degrees of freedom
## Residual deviance: 1136.3  on 987  degrees of freedom
## AIC: 1162.3
##
## Number of Fisher Scoring iterations: 4
```

e.

```
tree_model <- tree(recent_nicotine_use ~ ., data = train)
summary(tree_model)
```

```
##
## Classification tree:
## tree(formula = recent_nicotine_use ~ ., data = train)
## Variables actually used in tree construction:
## [1] "Country"   "SS"        "Education" "Cscore"    "Age"
## Number of terminal nodes:  7
## Residual mean deviance:  1.156 = 1148 / 993
## Misclassification error rate: 0.283 = 283 / 1000
```

```
plot(tree_model)
text(tree_model, pretty = 0)
```



f.

```
cv_model <- cv.tree(tree_model, FUN = prune.misclass)
cv_model$size
```

```
## [1] 7 5 4 3 2 1
```

```
cv_model$dev
```

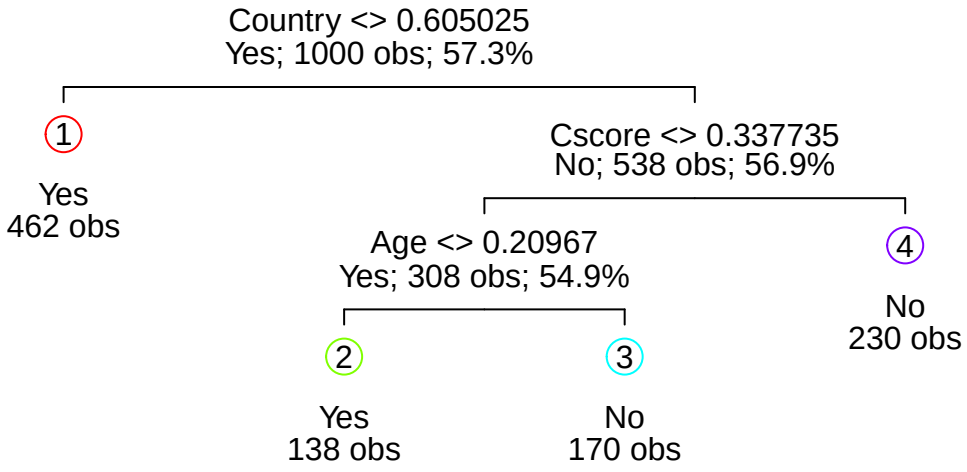
```
## [1] 307 307 301 357 359 433
```

```
best_s <- cv_model$size[which.min(cv_model$dev)]
best_s
```

```
## [1] 4
```

g.

```
pruned <- prune.misclass(tree_model, best = best_s)
draw.tree(pruned, nodeinfo = TRUE)
```



The variable that is split first is Age.

h.

```
tree_pred <- predict(pruned, newdata = test, type = "class")
confusion_matrix <- table(True = test$recent_nicotine_use,
                  Pred = tree_pred)
confusion_matrix
```

```
##      Pred
## True   No Yes
##   No  245 153
##   Yes 123 364
```

```
TP  <- confusion_matrix["Yes", "Yes"]
FN  <- confusion_matrix["Yes", "No"]
FP  <- confusion_matrix["No",  "Yes"]
TN  <- confusion_matrix["No",  "No"]
TPR <- TP / (TP + FN)
FPR <- FP / (FP + TN)
TPR
```

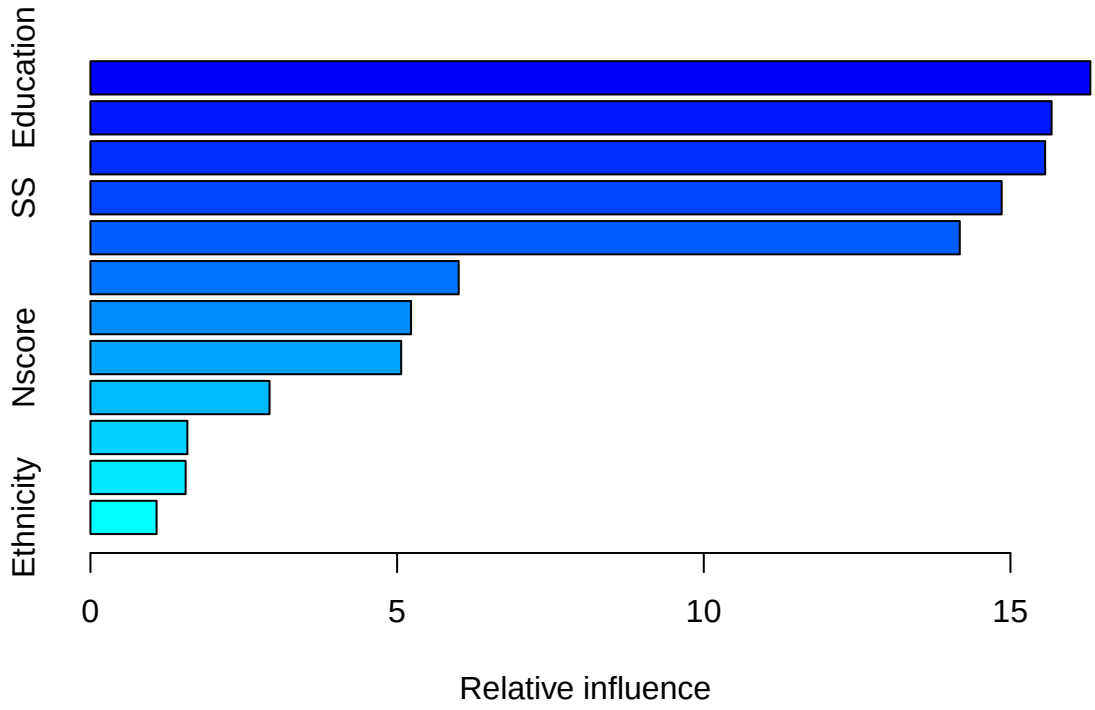
```
## [1] 0.7474333
```

```
FPR
```

```
## [1] 0.3844221
```

i.

```
train$recent_nicotine_use_num <- ifelse(train$recent_nicotine_use == "Yes", 1, 0)
test$recent_nicotine_use_num  <- ifelse(test$recent_nicotine_use  == "Yes", 1, 0)
boost_model <- gbm(
  recent_nicotine_use_num ~ . - recent_nicotine_use,
  data = train,
  distribution = "bernoulli",
  n.trees = 1000,
  shrinkage = 0.01
)
summary(boost_model)
```




```
##                 var   rel.inf
## Education Education 16.302814
## Country     Country 15.671163
## Age             Age 15.566256
## SS               SS 14.856700
## Cscore       Cscore 14.173801
## Oscore       Oscore  6.004052
```

```
## Escore       Escore  5.227225
## Nscore       Nscore  5.066641
## Gender       Gender  2.919801
## Ascore       Ascore  1.580787
## Impulsive Impulsive  1.552164
## Ethnicity Ethnicity  1.078596
```

Age seems to be the most important predictor

j.

```
rf <- randomForest(recent_nicotine_use ~ .,
                       data = train,
                       importance = TRUE)
rf
```

```
##
## Call:
##  randomForest(formula = recent_nicotine_use ~ ., data = train,      importance = TRUE)
##                Type of random forest: classification
##                      Number of trees: 500
## No. of variables tried at each split: 3
##
##         OOB estimate of  error rate: 0%
## Confusion matrix:
##      No Yes class.error
## No  427   0           0
## Yes   0 573           0
```

```
rf$err.rate[nrow(rf$err.rate), "OOB"]
```

```
## OOB
##   0
```

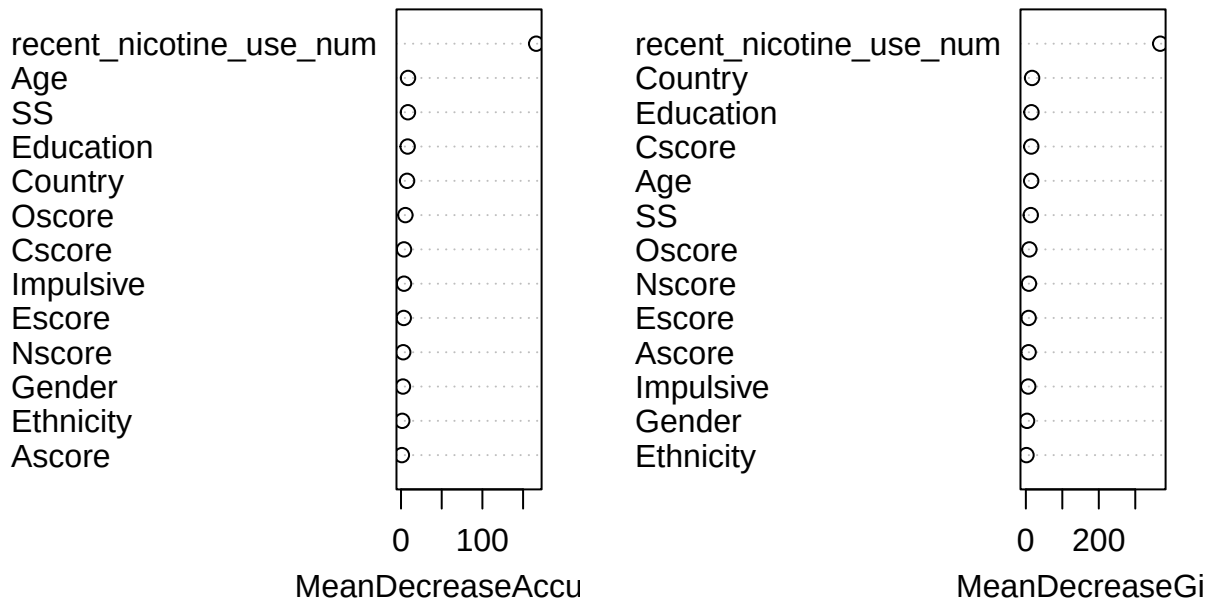
```
rf$mtry
```

```
## [1] 3
```

```
rf$ntree
```

```
## [1] 500
```

```
varImpPlot(rf)
```



Yes, the order of importance is similar, but they're not the same.

k.

```
boost_prob <- predict(boost_model, newdata = test, n.trees = 1000, type = "response")
boost_pred <- ifelse(boost_prob >= 0.20, "Yes", "No")
boost_pred <- factor(boost_pred, levels = c("No", "Yes"))
rf_prob <- predict(rf, newdata = test, type = "prob")[, "Yes"]
rf_pred <- ifelse(rf_prob >= 0.20, "Yes", "No")
rf_pred <- factor(rf_pred, levels = c("No", "Yes"))
boost_conf <- table(True = test$recent_nicotine_use, Pred = boost_pred)
rf_conf    <- table(True = test$recent_nicotine_use, Pred = rf_pred)
boost_conf
```

```
##      Pred
## True   No Yes
##   No   42 356
##   Yes   6 481
```

```
rf_conf
```

```
##      Pred
## True   No Yes
##   No  398   0
##   Yes   0 487
```

```r
sum(test$recent_nicotine_use[rf_pred == "Yes"] == "Yes") / sum(rf_pred == "Yes")
```

```
## [1] 1
```